

Python E-Course

Module 13 — Databases (SQLite)

Detailed Notes

Module Overview

Level: Applied

Duration: ~1.5 weeks

Prerequisites: Modules 1-12

What You'll Learn

- Read and write basic SQL (CRUD).
- Use the sqlite3 standard-library module.
- Use parameterized queries to prevent SQL injection.
- Manage transactions.
- Get a taste of an ORM (SQLAlchemy) for context.

Lessons

#	Lesson	Duration
1	SQL Primer	35 min
2	Connecting with sqlite3	30 min
3	CRUD Operations	35 min
4	Parameterized Queries & Safety	25 min
5	Transactions	25 min
6	Quick ORM Tour (SQLAlchemy)	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 14: Capstone Projects.

Lesson 1: SQL Primer

Module: 13 — Databases

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Read basic SQL: SELECT, INSERT, UPDATE, DELETE, CREATE TABLE.
- Filter, sort, and aggregate with WHERE, ORDER BY, GROUP BY.
- Understand tables, columns, rows, primary keys.

Prerequisites

- Modules 1-12

1. Why This Matters

SQL is a 50-year-old language that runs the world's data. Every backend developer needs to read it, write it, and reason about it. The good news: 80% of daily SQL is a handful of statements you can learn in an hour.

2. Concept

2.1 The data model

- A database holds tables.
- A table has columns (typed) and rows (records).
- A primary key uniquely identifies each row.

Example table users:

id	name	email	age
1	Maya	m@example.com	22
2	Sam	s@example.com	30

2.2 Create a table

```
CREATE TABLE users (  
  id      INTEGER PRIMARY KEY,  
  name    TEXT NOT NULL,  
  email   TEXT UNIQUE,  
  age     INTEGER  
);
```

Common SQLite types: INTEGER, REAL, TEXT, BLOB. SQLite is loosely typed — it accepts most data — but declare types anyway.

2.3 Insert rows

```
INSERT INTO users (name, email, age) VALUES ('Maya', 'm@example.com', 22);
```

You can omit id when it's INTEGER PRIMARY KEY — SQLite auto-fills.

2.4 Read with SELECT

```
SELECT * FROM users;
SELECT name, age FROM users;
SELECT name FROM users WHERE age > 21;
SELECT * FROM users ORDER BY age DESC LIMIT 3;
```

2.5 Update

```
UPDATE users SET age = 23 WHERE id = 1;
```

Always add a WHERE. UPDATE users SET age = 23; updates every row.

2.6 Delete

```
DELETE FROM users WHERE id = 1;
```

Same warning — without WHERE, you delete everything.

2.7 Aggregation

```
SELECT COUNT(*) FROM users;
SELECT AVG(age) FROM users;
SELECT MAX(age), MIN(age) FROM users;
```

2.8 GROUP BY

Count users per age:

```
SELECT age, COUNT(*) FROM users GROUP BY age;
```

2.9 Joins (preview)

If you have a posts table referencing users.id:

```
SELECT u.name, p.title
FROM users u
JOIN posts p ON p.user_id = u.id;
```

We'll keep joins light in this module.

3. Code Examples

We'll run real SQL in the next lesson. For now, study these and predict the output.

Example 1: Create + insert

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  age INTEGER
);

INSERT INTO users (name, age) VALUES ('Alice', 30);
INSERT INTO users (name, age) VALUES ('Bob', 25);
INSERT INTO users (name, age) VALUES ('Carol', 28);
```

```
SELECT * FROM users;
```

Expected result:

```
1 | Alice | 30
2 | Bob   | 25
3 | Carol | 28
```

Example 2: Filter and sort

```
SELECT name, age FROM users WHERE age >= 28 ORDER BY age DESC;
```

Expected:

```
Alice | 30
Carol | 28
```

Example 3: Count and average

```
SELECT COUNT(*) AS total, AVG(age) AS avg_age FROM users;
```

Expected:

```
total | avg_age
3      | 27.67
```

Example 4: Update + delete

```
UPDATE users SET age = 31 WHERE name = 'Alice';
DELETE FROM users WHERE age < 28;
SELECT * FROM users;
```

Expected (after both):

```
1 | Alice | 31
3 | Carol | 28
```

4. Common Mistakes

Mistake	What Happens	Fix
Missing WHERE on UPDATE/DELETE	Affects every row	Always include WHERE
Single-quoted column names	"name" (double) or unquoted; single is for strings	Don't quote column names; or use double quotes/backticks per DB
= vs ==	SQL uses single =	WHERE id = 1
Forgetting ;	Some interactive shells need it	Add ;
Inserting wrong number of values	Error	Match columns and values

5. Practice Tasks

- Write a CREATE TABLE for books(id, title, author, year).

- Write an INSERT for one book.
- Write a SELECT for books published after 2000, sorted by year DESC.

6. Key Takeaways

- Tables are rows + typed columns; primary key uniquely identifies rows.
- SELECT, INSERT, UPDATE, DELETE are the four bread-and-butter statements.
- Always use WHERE on UPDATE/DELETE.
- COUNT, AVG, MIN, MAX, GROUP BY for aggregations.

7. What's Next

Connect Python to a SQLite database and run the SQL above for real.

Lesson 2: Connecting with sqlite3

Module: 13 — Databases

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Connect to a SQLite database file.
- Execute SQL via a cursor.
- Iterate over query results.

Prerequisites

- Module 13 Lesson 1

1. Why This Matters

SQLite is a tiny, file-based database used in millions of apps — your phone, browser, OS. The Python sqlite3 module is in the standard library; no install, no server, no setup. Perfect first database.

2. Concept

2.1 Open a connection

```
import sqlite3
conn = sqlite3.connect("my.db")
```

If my.db doesn't exist, it's created. :memory: opens an ephemeral in-memory database — great for tests.

2.2 Cursor

You execute SQL via a cursor:

```
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT)")
cur.execute("INSERT INTO users (name) VALUES ('Maya')")
conn.commit()
```

2.3 Commit and close

Changes aren't saved until conn.commit(). Always close when done — or use with:

```
with sqlite3.connect("my.db") as conn:
    conn.execute("INSERT INTO users (name) VALUES ('Sam')")
    # conn commits automatically on success
```

2.4 Query and fetch

```
cur = conn.execute("SELECT id, name FROM users")
print(cur.fetchall())          # list of tuples
# or
```

```
for row in conn.execute("SELECT id, name FROM users"):
    print(row)
```

Method	Returns
fetchone()	next row or None
fetchmany(n)	up to n rows
fetchall()	all remaining rows
iterate	one row at a time

2.5 Rows as dicts

By default, rows are tuples — row[0], row[1]. Switch to sqlite3.Row for name access:

```
conn.row_factory = sqlite3.Row
for row in conn.execute("SELECT * FROM users"):
    print(row["name"])
```

2.6 executemany for bulk inserts

```
data = [("Maya",), ("Sam",), ("Lee",)]
conn.executemany("INSERT INTO users (name) VALUES (?)", data)
```

2.7 lastrowid

After INSERT, get the new row's ID:

```
cur = conn.execute("INSERT INTO users (name) VALUES (?)", ("Maya",))
print(cur.lastrowid)
```

3. Code Examples

Example 1: Create + insert + select

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.execute("""
        CREATE TABLE users (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            age INTEGER
        )
    """)
    conn.execute("INSERT INTO users (name, age) VALUES ('Alice', 30)")
    conn.execute("INSERT INTO users (name, age) VALUES ('Bob', 25)")
    for row in conn.execute("SELECT * FROM users"):
        print(row)
```

Expected Output:

```
(1, 'Alice', 30)
(2, 'Bob', 25)
```


Example 2: Row factory for dict-like rows

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.row_factory = sqlite3.Row
    conn.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)")
    conn.execute("INSERT INTO users (name) VALUES ('Maya')")
    row = conn.execute("SELECT * FROM users WHERE id = 1").fetchone()
    print(row["name"])
    print(dict(row))
```

Expected Output:

```
Maya
{'id': 1, 'name': 'Maya'}
```

Example 3: executemany

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE items (id INTEGER PRIMARY KEY, label TEXT)")
    conn.executemany(
        "INSERT INTO items (label) VALUES (?)",
        [("apple",), ("banana",), ("cherry",)]
    )
    rows = conn.execute("SELECT COUNT(*) FROM items").fetchone()
    print("count:", rows[0])
```

Expected Output: count: 3

Example 4: lastrowid

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, name TEXT)")
    cur = conn.execute("INSERT INTO t (name) VALUES (?)", ("first",))
    print("id:", cur.lastrowid)
```

Expected Output: id: 1

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting commit()	Changes lost	Use with (auto-commits on success)
Mixing connections from different threads	OperationalError	One connection per thread or use check_same_thread=False
Not closing connections	File locked, leaks	Use with or call .close()
String concatenation for values	SQL injection (next lesson)	Use ? placeholders

5. Practice Tasks

- Create a SQLite DB file notes.db with a notes(id, body) table. Insert 2 notes. Print all rows.
- Switch to row_factory = sqlite3.Row and re-read the same data; access fields by name.
- Use executemany to bulk-insert 5 rows.

6. Key Takeaways

- sqlite3.connect(path) — file or :memory:.
- Execute SQL via conn.execute(...) or a cursor.
- Use with sqlite3.connect(...) for auto-commit and cleanup.
- sqlite3.Row for dict-like row access.

7. What's Next

Full CRUD — create, read, update, delete — from Python.

Lesson 3: CRUD Operations

Module: 13 — Databases

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Perform Create, Read, Update, Delete from Python.
- Wrap each operation in a small function.
- Use RETURNING and lastrowid.

Prerequisites

- Module 13 Lesson 2

1. Why This Matters

The four CRUD operations are 90% of database work. Building a clean Python wrapper around them gives you a tiny "data layer" your other code can call without knowing SQL.

2. Concept

2.1 Design choice

For a small app, write per-operation functions:

```
def create_user(conn, name, email): ...
def get_user(conn, user_id): ...
def update_user(conn, user_id, *, name=None, email=None): ...
def delete_user(conn, user_id): ...
def list_users(conn): ...
```

Each takes the connection as the first arg. This makes them easy to test.

2.2 Create (INSERT)

```
def create_user(conn, name, email):
    cur = conn.execute(
        "INSERT INTO users (name, email) VALUES (?, ?)",
        (name, email),
    )
    return cur.lastrowid
```

2.3 Read (SELECT)

Single row:

```
def get_user(conn, user_id):
    row = conn.execute(
        "SELECT * FROM users WHERE id = ?", (user_id,)
    ).fetchone()
    return dict(row) if row else None
```

Many rows:

```
def list_users(conn):
    return [dict(row) for row in conn.execute("SELECT * FROM users")]
```

(Set `conn.row_factory = sqlite3.Row` to enable dict conversion.)

2.4 Update

```
def update_user(conn, user_id, *, name=None, email=None):
    sets, values = [], []
    if name is not None:
        sets.append("name = ?")
        values.append(name)
    if email is not None:
        sets.append("email = ?")
        values.append(email)
    if not sets:
        return 0
    values.append(user_id)
    cur = conn.execute(
        f"UPDATE users SET {'', '.join(sets)} WHERE id = ?",
        values,
    )
    return cur.rowcount
```

rowcount tells you how many rows were affected.

2.5 Delete

```
def delete_user(conn, user_id):
    cur = conn.execute("DELETE FROM users WHERE id = ?", (user_id,))
    return cur.rowcount
```

2.6 RETURNING (SQLite ≥ 3.35)

```
row = conn.execute(
    "INSERT INTO users (name) VALUES (?) RETURNING id, name",
    ("Maya",),
).fetchone()
```

A cleaner alternative to `lastrowid` for new APIs.

2.7 Schema initialization helper

```
def init_schema(conn):
    conn.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            email TEXT UNIQUE
        );
    """)
```

`executescript` runs multiple statements separated by `;`.

3. Code Examples

Example 1: End-to-end CRUD

```
import sqlite3

def init(conn):
    conn.executescript("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        email TEXT UNIQUE
    );
    """)

def create_user(conn, name, email):
    cur = conn.execute(
        "INSERT INTO users (name, email) VALUES (?, ?)",
        (name, email),
    )
    return cur.lastrowid

def list_users(conn):
    return [dict(r) for r in conn.execute("SELECT * FROM users")]

def get_user(conn, uid):
    row = conn.execute("SELECT * FROM users WHERE id = ?", (uid,)).fetchone()
    return dict(row) if row else None

def update_email(conn, uid, email):
    cur = conn.execute("UPDATE users SET email = ? WHERE id = ?", (email, uid))
    return cur.rowcount

def delete_user(conn, uid):
    cur = conn.execute("DELETE FROM users WHERE id = ?", (uid,))
    return cur.rowcount

with sqlite3.connect(":memory:") as conn:
    conn.row_factory = sqlite3.Row
    init(conn)
    a = create_user(conn, "Alice", "a@x.com")
    b = create_user(conn, "Bob", "b@x.com")
    print("list:", list_users(conn))
    update_email(conn, a, "alice@x.com")
    print("get  :", get_user(conn, a))
    print("del  :", delete_user(conn, b))
    print("list :", list_users(conn))
```

Expected Output:

```
list: [{'id': 1, 'name': 'Alice', 'email': 'a@x.com'}, {'id': 2, 'name': 'Bob',
'email': 'b@x.com'}]
get  : {'id': 1, 'name': 'Alice', 'email': 'alice@x.com'}
```

```
del : 1
list : [{'id': 1, 'name': 'Alice', 'email': 'alice@x.com'}]
```

Example 2: rowcount on bulk update

```
import sqlite3
with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE u (id INTEGER PRIMARY KEY, age INTEGER)")
    conn.executemany("INSERT INTO u (age) VALUES (?)", [(10,), (20,), (30,)])
    cur = conn.execute("UPDATE u SET age = age + 1 WHERE age >= 20")
    print("rows updated:", cur.rowcount)
```

Expected Output: rows updated: 2

4. Common Mistakes

Mistake	What Happens	Fix
Building SQL with f-strings + values	SQL injection	Use ? placeholders for values
WHERE missing on UPDATE/DELETE	Hits all rows	Always specify
Treating fetchone() as never None	Crashes when no rows	Check before indexing
Forgetting to commit on a raw connection	Changes lost	Use with or call .commit()

5. Practice Tasks

- Build books(id, title, author, year). Add CRUD helpers.
- Insert 5 books; query those after 2000.
- Update the year of one row and verify with rowcount.

6. Key Takeaways

- Wrap each operation in a small function.
- Use ? placeholders for values; build column lists from validated keys only.
- rowcount tells you how many rows you changed.
- RETURNING (3.35+) is a clean alternative to lastrowid.

7. What's Next

Why we use ? placeholders — parameterized queries and the SQL injection threat.

Lesson 4: Parameterized Queries & Safety

Module: 13 — Databases

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Recognize SQL injection.
- Always use parameterized queries (?, :name).
- Safely include dynamic identifiers (column / table names).

Prerequisites

- Module 13 Lesson 3

1. Why This Matters

SQL injection is one of the oldest and most dangerous vulnerabilities — it's still in OWASP Top 10. The fix is trivial: never concatenate user input into SQL. Use placeholders. Once it's a habit, you can't get this wrong.

2. Concept

2.1 The bug

```
# DON'T
cur.execute(f"SELECT * FROM users WHERE name = '{name}'")
```

If name == "x' OR '1'='1", the query becomes:

```
SELECT * FROM users WHERE name = 'x' OR '1'='1'
```

...which returns all users. Worse payloads can drop tables or steal data.

2.2 The fix — ? placeholders

```
cur.execute("SELECT * FROM users WHERE name = ?", (name,))
```

The database driver handles quoting and escaping. The user input is data, never executable SQL.

2.3 Named placeholders — :name

```
cur.execute(
    "INSERT INTO users (name, age) VALUES (:name, :age)",
    {"name": "Maya", "age": 22},
)
```

Useful with more fields — order doesn't matter.

2.4 What placeholders can't do

Placeholders only work for values — not for table names, column names, or SQL keywords. If you need a dynamic table/column name (rare), whitelist it:

```
ALLOWED_SORT = {"name", "age", "created_at"}

def list_users(conn, sort_by):
    if sort_by not in ALLOWED_SORT:
        raise ValueError("invalid sort column")
    return conn.execute(f"SELECT * FROM users ORDER BY {sort_by}").fetchall()
```

The whitelist guarantees the value came from your code, not user input.

2.5 executemany for bulk

Still uses placeholders:

```
conn.executemany(
    "INSERT INTO users (name) VALUES (?)",
    [("Maya",), ("Sam",), ("Lee",)],
)
```

3. Code Examples

Example 1: Vulnerable vs safe

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)")
    conn.execute("INSERT INTO users (name) VALUES ('Maya')")
    conn.execute("INSERT INTO users (name) VALUES ('Sam')")

    # Vulnerable
    bad_input = "x' OR '1'='1"
    rows = conn.execute(f"SELECT * FROM users WHERE name = '{bad_input}'").fetchall()
    print("vulnerable returns:", len(rows), "rows")

    # Safe
    rows = conn.execute("SELECT * FROM users WHERE name = ?",
                        (bad_input,)).fetchall()
    print("safe returns      :", len(rows), "rows")
```

Expected Output:

```
vulnerable returns: 2 rows
safe returns      : 0 rows
```

Example 2: Named params

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE u (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")
    conn.execute(
```



```

        "INSERT INTO u (name, age) VALUES (:name, :age)",
        {"name": "Maya", "age": 22},
    )
    row = conn.execute("SELECT * FROM u WHERE id = :id", {"id": 1}).fetchone()
    print(row)

```

Expected Output: (1, 'Maya', 22)

Example 3: Whitelisted dynamic column

```

ALLOWED = {"name", "age"}

def safe_sort(conn, column):
    if column not in ALLOWED:
        raise ValueError(f"bad column: {column}")
    return conn.execute(f"SELECT * FROM u ORDER BY {column}").fetchall()

```

(No untrusted SQL gets in.)

Example 4: Bulk insert

```

import sqlite3
with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE t (label TEXT)")
    conn.executemany("INSERT INTO t (label) VALUES (?)",
        [("a",), ("b",), ("c",)])
    print(conn.execute("SELECT COUNT(*) FROM t").fetchone()[0])

```

Expected Output: 3

4. Common Mistakes

Mistake	What Happens	Fix
f-string with user input	SQL injection	? placeholder
(name) instead of (name,)	Treated as wrong type	Use a 1-tuple (name,)
Placeholder for table name	Doesn't work	Whitelist + f-string only after validation
Forgetting to commit	Changes lost	Use with

5. Practice Tasks

- Rewrite an f-string query as a parameterized one.
- Use named :name placeholders to insert a user.
- Implement `safe_sort(conn, column)` with a whitelist of valid columns.

6. Key Takeaways

- Never concatenate user input into SQL.
- Use ? (positional) or :name (named) placeholders for values.
- Whitelist for any dynamic table/column names.

7. What's Next

Transactions — make multi-step changes atomic.

Lesson 5: Transactions

Module: 13 — Databases

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Explain why transactions matter.
- Use the connection as a context manager to group statements.
- Manually commit and rollback.

Prerequisites

- Module 13 Lesson 4

1. Why This Matters

The classic example: transferring money. Subtract from one account, add to another. If the second step fails after the first, you have inconsistent state. Transactions make the pair succeed or fail together — all-or-nothing.

2. Concept

2.1 What a transaction is

A unit of work containing one or more statements. Either all commit or all roll back. Defined by ACID properties: Atomic, Consistent, Isolated, Durable.

2.2 sqlite3 and implicit transactions

By default, sqlite3 opens a transaction before any data-modifying statement and commits when you call `conn.commit()`. If you with `sqlite3.connect(...)` and the block raises, the transaction rolls back automatically.

2.3 The with pattern

```
import sqlite3
conn = sqlite3.connect("bank.db")
try:
    with conn:
        conn.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?", (100,
1))
        conn.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?", (100,
2))
        # if both lines succeed: commit
except Exception as e:
    print("rolled back:", e)
finally:
    conn.close()
```

If either UPDATE raises (or you raise manually), the connection rolls back — leaving balances untouched.

2.4 Manual control

```
conn = sqlite3.connect("bank.db")
try:
    conn.execute("BEGIN")
    conn.execute(...)
    conn.execute(...)
    conn.commit()
except Exception:
    conn.rollback()
    raise
```

Use this only when you need fine control. with conn: is enough most of the time.

2.5 Common patterns

Transfer:

```
def transfer(conn, src, dst, amount):
    with conn:
        # Check balance first
        balance = conn.execute(
            "SELECT balance FROM accounts WHERE id = ?", (src,)
        ).fetchone()[0]
        if balance < amount:
            raise ValueError("insufficient funds")
        conn.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?",
            (amount, src))
        conn.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?",
            (amount, dst))
```

If the second UPDATE somehow fails or the raise fires, the first UPDATE rolls back.

2.6 Isolation in SQLite

SQLite supports serializable isolation by default — strong but uses file-level locking. Fine for single-process apps. Multi-writer concurrency needs care; for that, look at PostgreSQL or MySQL later.

3. Code Examples

Example 1: Successful transfer

```
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.executescript("""
        CREATE TABLE accounts (id INTEGER PRIMARY KEY, name TEXT, balance INTEGER);
        INSERT INTO accounts (name, balance) VALUES ('Alice', 1000);
        INSERT INTO accounts (name, balance) VALUES ('Bob', 500);
    """)
```

```

def transfer(conn, src, dst, amount):
    with conn:
        bal = conn.execute("SELECT balance FROM accounts WHERE id = ?",
(src,)).fetchone()[0]
        if bal < amount:
            raise ValueError("insufficient")
        conn.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?",
(amount, src))
        conn.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?",
(amount, dst))

transfer(conn, 1, 2, 200)
for row in conn.execute("SELECT * FROM accounts"):
    print(row)

```

Expected Output:

```

(1, 'Alice', 800)
(2, 'Bob', 700)

```

Example 2: Rollback on error

```

import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.executescript("""
CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance INTEGER);
INSERT INTO accounts (balance) VALUES (100);
INSERT INTO accounts (balance) VALUES (100);
""")

    try:
        with conn:
            conn.execute("UPDATE accounts SET balance = balance - 50 WHERE id = 1")
            raise RuntimeError("simulated crash")
    except RuntimeError as e:
        print("error:", e)

    # Verify nothing changed
    for row in conn.execute("SELECT * FROM accounts"):
        print(row)

```

Expected Output:

```

error: simulated crash
(1, 100)
(2, 100)

```

Example 3: Manual commit/rollback

```

import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t (x INTEGER)")

try:

```

```

conn.execute("BEGIN")
conn.execute("INSERT INTO t VALUES (1)")
conn.execute("INSERT INTO t VALUES (2)")
conn.commit()
except Exception:
    conn.rollback()
    raise

print(conn.execute("SELECT COUNT(*) FROM t").fetchone()[0])
conn.close()

```

Expected Output: 2

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting with conn: and commit()	Changes lost	Use with
Holding a transaction open across slow operations	Other writers blocked	Keep transactions short
Catching the exception inside with conn:	Prevents rollback	Let it propagate or re-raise
Confusing with sqlite3.connect() (closes conn) with with conn: (transaction)	Different scopes	Both are useful; understand the difference

5. Practice Tasks

- Write a transfer(conn, src, dst, amount) that uses with conn:. Test successful and failed cases.
- Wrap two INSERTs in a transaction; raise after the first; verify both roll back.
- Use a manual BEGIN/commit/rollback and compare to the with form.

6. Key Takeaways

- Transactions = all-or-nothing groups of statements.
- with conn: auto-commits on success, rolls back on exception.
- Keep transactions short.
- Don't swallow exceptions inside the with block.

7. What's Next

A short tour of SQLAlchemy — an ORM — to see what comes after raw sqlite3.

Lesson 6: Quick ORM Tour (SQLAlchemy)

Module: 13 — Databases

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Explain what an ORM is and when to reach for one.
- Define models with SQLAlchemy 2.0.
- Run basic queries with Session.

Prerequisites

- Module 13 Lesson 5, Module 10 (venv/pip).

1. Why This Matters

Writing raw SQL gets tedious as schemas grow. An ORM (Object-Relational Mapper) lets you work with Python objects and have SQL generated for you. SQLAlchemy is the de facto standard in Python. We won't go deep — but you should know it exists before Module 14.

2. Concept

2.1 Install

```
pip install "sqlalchemy>=2"
```

2.2 The pieces

- Engine: the connection factory + dialect.
- Models: Python classes mapped to tables.
- Session: a unit of work; you add, query, and commit through it.

2.3 A minimal example

```
from sqlalchemy import create_engine, String, Integer
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    age: Mapped[int | None] = mapped_column(Integer, nullable=True)

engine = create_engine("sqlite:///app.db", echo=False)
Base.metadata.create_all(engine)

with Session(engine) as session:
```

```

session.add(User(name="Maya", age=22))
session.add(User(name="Sam", age=30))
session.commit()

for u in session.query(User).order_by(User.name):
    print(u.id, u.name, u.age)

```

2.4 Queries

```

# All rows
session.query(User).all()

# Filter
session.query(User).filter(User.age >= 25).all()

# Single row by primary key
session.get(User, 1)

# Count
session.query(User).count()

```

In modern SQLAlchemy you can also use `session.execute(select(User).where(...))` — both styles are valid; `query()` reads easily for beginners.

2.5 Update / delete

```

user = session.get(User, 1)
user.age = 23
session.commit()

session.delete(user)
session.commit()

```

You mutate the object; the session generates the SQL.

2.6 When to use ORM vs raw SQL

Use ORM	Use raw SQL
Many tables with relationships	Quick scripts
Web app with many CRUD endpoints	Single-table operations
Want object-oriented code	Need very specific/optimized SQL
Team will refactor schemas	Performance-critical batch jobs

For this course we stop here — Module 13's assignment uses raw `sqlite3`. SQLAlchemy is on your roadmap for after.

3. Code Examples

Example 1: Full mini-app

```

from sqlalchemy import create_engine, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase):
    pass

```

```

class Task(Base):
    __tablename__ = "tasks"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    done: Mapped[bool] = mapped_column(default=False)

engine = create_engine("sqlite:///memory:")
Base.metadata.create_all(engine)

with Session(engine) as session:
    session.add_all([
        Task(title="Buy groceries"),
        Task(title="Email Maya"),
        Task(title="Read book", done=True),
    ])
    session.commit()

    pending = session.query(Task).filter_by(done=False).all()
    for t in pending:
        print(t.id, t.title)

```

Expected Output:

```

1 Buy groceries
2 Email Maya

```

Example 2: Update

```

from sqlalchemy import create_engine, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase): pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String)
    age: Mapped[int] = mapped_column(Integer)

engine = create_engine("sqlite:///memory:")
Base.metadata.create_all(engine)

with Session(engine) as s:
    s.add(User(name="Maya", age=22))
    s.commit()

    u = s.get(User, 1)
    u.age = 23
    s.commit()

    print(s.get(User, 1).age)

```

Expected Output: 23

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting commit()	Changes not saved	Always commit (or use with session.begin())
Using ORM for tiny scripts	Overkill	Stick with sqlite3
Loading huge tables with .all()	Memory blowup	Query lazily / paginate
Mixing different sessions	Confused state	One session per logical operation

5. Practice Tasks

- Define a Book model with id/title/author. Add 3 books and list them.
- Update the author of one book.
- Delete a book and verify it's gone.

6. Key Takeaways

- An ORM maps Python classes to tables.
- SQLAlchemy is the standard for Python.
- Use ORM for larger schemas; raw sqlite3 for small scripts.

7. What's Next

End of Module 13. Next: Module 14 — Capstone Projects, where you put everything together.

Module 13 — Exercises

18 exercises.

13.1 SQL primer

- (E) Write SQL to create books(id, title, author, year).
- (M) Insert 3 books; select all sorted by year DESC.
- (H) Count books per author using GROUP BY.

13.2 sqlite3

- (E) Open :memory;; create notes(id, body); insert 2 rows; select all.
- (M) Use sqlite3.Row row factory; access fields by name.
- (H) Bulk-insert 10 rows with executemany.

13.3 CRUD

- (E) Write create_book and list_books functions.
- (M) Add update_book_year(book_id, year) returning rowcount.
- (H) Add delete_book_by_author(author); return how many deleted.

13.4 Parameterized

- (E) Rewrite f"... WHERE name = '{name}'" using ?.
- (M) Use :name named params for an insert.
- (H) Implement a whitelisted sort_by for ORDER BY.

13.5 Transactions

- (E) Wrap two inserts in with conn: and confirm both committed.
- (M) Raise after the first insert; verify both rolled back.
- (H) Write transfer(conn, src, dst, amount); cover insufficient-funds case.

13.6 ORM

- (E) Define a Task(id, title, done) SQLAlchemy model and create the table.
- (M) Insert 3 tasks and query the pending ones.
- (H) Update one task, delete another, and confirm the state.

Module 13 — Quiz

10 MCQs.

Q1

Which statement deletes all rows from users (USE CAREFULLY)? A. DELETE FROM users B. DROP TABLE users C. TRUNCATE users D. DELETE users.*

Answer: A — but always include a WHERE in practice. (L1)

Q2

Which Python module is built-in for SQLite? A. pysqlite B. sqlite3 C. sqllitedb D. dbi

Answer: B (L2)

Q3

What does cur.lastrowid return? A. The last query string B. The ID of the newly inserted row C. The number of rows affected D. The cursor's position

Answer: B (L2)

Q4

Which placeholder style is correct for sqlite3? A. WHERE id = %s B. WHERE id = ? C. WHERE id = \$1 D. WHERE id = {0}

Answer: B (L4)

Q5

Why use ? placeholders? A. Faster execution B. Prevent SQL injection C. Better readability D. Required by SQL spec

Answer: B (L4)

Q6

Placeholders can substitute: A. Values only B. Values and column names C. Values and table names D. Anything

Answer: A — use a whitelist for identifiers. (L4)

Q7

What does with sqlite3.connect(path) as conn: do at block exit? A. Closes the connection B. Commits the current transaction (or rolls back on exception) C. Both D. Nothing

Answer: B — note: it commits/rolls back; you still need conn.close() or another with. (L2/L5)

Q8

What's rowcount on an UPDATE/DELETE? A. The new row count B. Number of rows affected C. Cursor position D. Always 1

Answer: B (L3)

Q9

Which is an advantage of an ORM? A. Always faster than raw SQL B. Eliminates SQL knowledge C. Lets you work with objects; generates SQL for you D. Works without a database

Answer: C (L6)

Q10

When are transactions especially important? A. Reads only B. Multi-statement updates that must succeed together C. Performance tuning D. Schema migrations only

Answer: B (L5)

Module 13 — Assignment: Expense Tracker Database

Estimated time: 2-3 hours

Combines: Module 13 + Modules 4, 7, 8-10

Brief

Build a CLI expense tracker backed by SQLite. Users can add expenses (amount, category, note, date), list them with filters, get a category-wise summary, and reset.

Requirements

Schema (init on first run):

```
CREATE TABLE IF NOT EXISTS expenses (  
    id INTEGER PRIMARY KEY,  
    amount REAL NOT NULL,  
    category TEXT NOT NULL,  
    note TEXT,  
    spent_at TEXT NOT NULL    -- ISO date string  
);  
  
CREATE INDEX IF NOT EXISTS idx_category ON expenses(category);
```

Functions (in db.py):

- `connect(path)` — returns a connection with `row_factory = sqlite3.Row`.
- `init_schema(conn)`.
- `add_expense(conn, amount, category, note, spent_at)` → new id.
- `list_expenses(conn, *, category=None, since=None, until=None, limit=None)` → list of rows.
- `summary(conn)` → list of (category, total) tuples sorted by total desc.
- `delete_expense(conn, id)` → rowcount.
- `reset(conn)` — drops all rows after confirmation.

A CLI (cli.py) with commands:

- `add AMOUNT CATEGORY [--note ...] [--date YYYY-MM-DD]`
- `list [--category C] [--since D] [--until D] [--limit N]`
- `summary`
- `delete ID`

Constraints

- All SQL via parameterized queries.
- Wrap multi-statement ops in with conn:.
- Custom `ExpenseError` for domain errors.
- Use `argparse` from M10 for the CLI.

Expected Behavior (sample)

```
$ python -m expenses add 450 food --note "lunch"
added id=1
```

```
$ python -m expenses add 1200 transport
added id=2
```

```
$ python -m expenses list
1 | 2026-05-23 | food      | ₹ 450.00 | lunch
2 | 2026-05-23 | transport | ₹ 1200.00 |
```

```
$ python -m expenses summary
transport ₹ 1200.00
food      ₹ 450.00
TOTAL     ₹ 1650.00
```

Grading Rubric

Criterion	Weight
Schema + indexes correct	15%
All CRUD functions	30%
Parameterized queries	15%
CLI works	20%
Style, exceptions	20%

Submission

Folder layout:

```
expenses/
├── pyproject.toml
├── README.md
├── expenses/
│   ├── __init__.py
│   ├── __main__.py
│   ├── db.py
│   ├── cli.py
│   └── errors.py
```

Module 13 — Mini Project: Contact Book v2

Overview

A SQLite-backed contact book. Stores name, phone, email per contact. Add, list, find, update, delete from the command line. All persisted between runs.

Concepts Used

- sqlite3 (M13 L2-L3)
- Parameterized queries (M13 L4)
- Transactions via with conn: (M13 L5)
- Functions, custom exceptions (M4, M8)

Features

- add NAME PHONE EMAIL
- list (all)
- find QUERY (case-insensitive on name/phone/email)
- update ID --name X --phone X --email X
- delete ID
- Persists to contacts.db.

Starter Code

contacts.py:

```
import sqlite3
import sys
from pathlib import Path

DB = Path("contacts.db")

def connect():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    conn.execute("""
    CREATE TABLE IF NOT EXISTS contacts (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        phone TEXT,
        email TEXT
    )""")
    return conn

def add(conn, name, phone, email):
    with conn:
        cur = conn.execute(
            "INSERT INTO contacts (name, phone, email) VALUES (?, ?, ?)",
            (name, phone, email),
        )
```

```

        return cur.lastrowid

def list_all(conn):
    return [dict(r) for r in conn.execute("SELECT * FROM contacts ORDER BY name")]

def find(conn, q):
    like = f"%{q.lower()}%"
    rows = conn.execute("""
        SELECT * FROM contacts
        WHERE LOWER(name) LIKE ?
            OR LOWER(phone) LIKE ?
            OR LOWER(email) LIKE ?
        ORDER BY name
    """, (like, like, like))
    return [dict(r) for r in rows]

def update(conn, cid, **fields):
    fields = {k: v for k, v in fields.items() if v is not None}
    if not fields:
        return 0
    sets = ", ".join(f"{k} = ?" for k in fields)
    values = list(fields.values()) + [cid]
    with conn:
        cur = conn.execute(f"UPDATE contacts SET {sets} WHERE id = ?", values)
    return cur.rowcount

def delete(conn, cid):
    with conn:
        cur = conn.execute("DELETE FROM contacts WHERE id = ?", (cid,))
    return cur.rowcount

def print_contact(c):
    print(f"{c['id']:>3} | {c['name']:<20} | {c.get('phone') or '-':<14} | {c.get('email') or '-'}")

def run():
    conn = connect()
    args = sys.argv[1:]
    if not args:
        print("usage: contacts add|list|find|update|delete ...")
        return
    cmd, *rest = args
    match cmd:
        case "add":
            name, phone, email = (rest + ["", ""])[0:3]
            cid = add(conn, name, phone or None, email or None)
            print(f"added id={cid}")
        case "list":
            for c in list_all(conn):
                print_contact(c)
        case "find":
            for c in find(conn, " ".join(rest)):
                print_contact(c)

```



```

        case "delete":
            print("deleted" if delete(conn, int(rest[0])) else "not found")
        case _:
            print("unknown command")
    conn.close()

if __name__ == "__main__":
    run()

```

Expected Interaction

```

$ python contacts.py add "Maya Sharma" 9876543210 maya@x.com
added id=1
$ python contacts.py add "Sam Lee" 9123456780 sam@x.com
added id=2
$ python contacts.py list
 1 | Maya Sharma          | 9876543210      | maya@x.com
 2 | Sam Lee               | 9123456780      | sam@x.com
$ python contacts.py find lee
 2 | Sam Lee               | 9123456780      | sam@x.com
$ python contacts.py delete 1
deleted

```

Extension Challenges

- Add an update subcommand using argparse.
- Add an export command that writes a CSV.
- Add unique constraints (name + phone) and handle IntegrityError.

Grading Rubric

Criterion	Weight
Schema + CRUD work	35%
Parameterized queries	20%
Search is case-insens.	15%
Persists between runs	15%
Style + functions	15%